

VHDL & FPGA Over View

Module Convenor: Prof. T. Nandha Kumar

Email: nandhakumaar.t@nottingham.edu.my

Department of Electrical & Electronic Engineering

University of Nottingham Malaysia

Objectives

After completing this Lecture, you will be able to:

- Understand the role of HDL in IC design Industries
- Understand the ASIC and FPGA flow
- Understand the basic architecture of Spartan FPGA
- Understand the basic use of simulator



New Learning Outcomes 2023/24

- LO1: Use synthesizable VHDL statements to design medium-complex digital circuits
- LO2: Design VHDL testbench codes for medium complex digital circuits and simulate VHDL codes using a commercial simulator
- LO3: Use synthesizable VHDL statements to design an Industrial bus protocol
- LO4: Design VHDL testbench codes for an Industrial bus protocol and simulate VHDL codes using a commercial simulator
- LO5: Synthesise VHDL codes and then explore implementation into a commercial development board

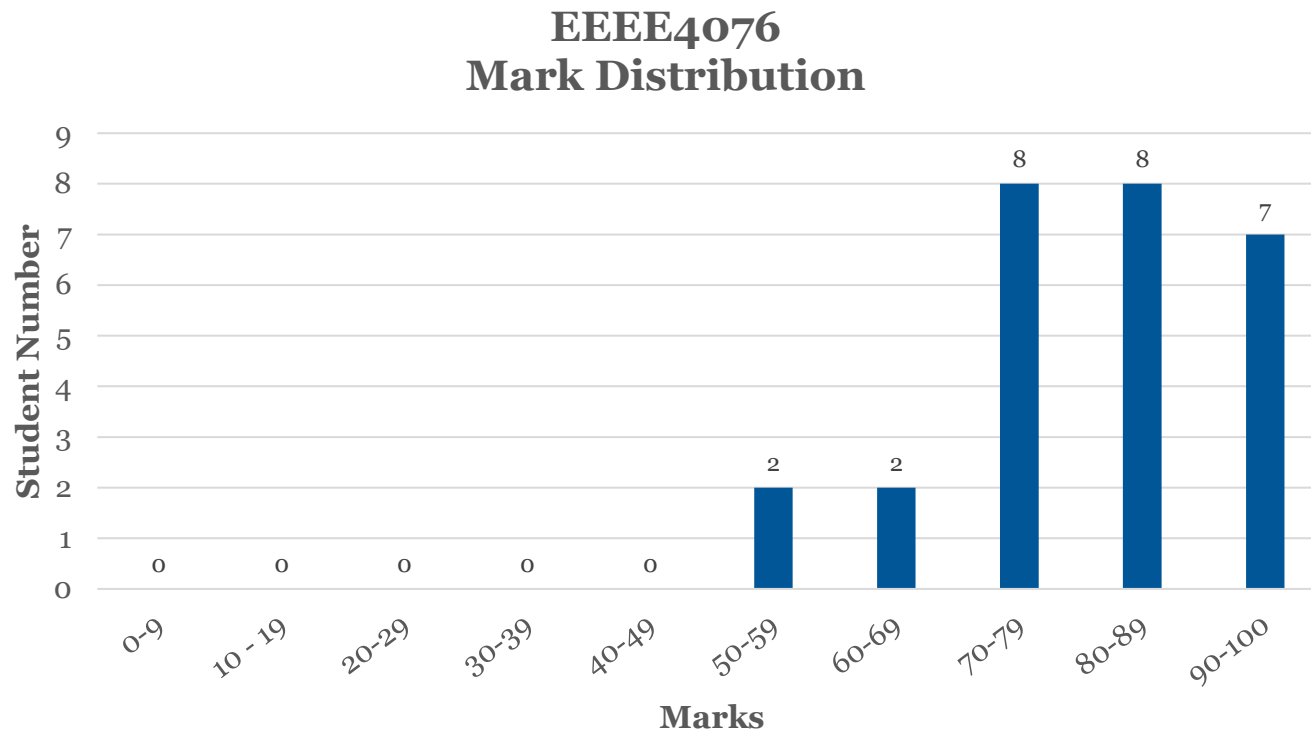


Assessment Results:

Total Students: 27 Total Absent: 0 Fail: 0

Average: 79.5 Median: 80.9

Any scaling has been applied: No



Types of Assessment:

2025-26

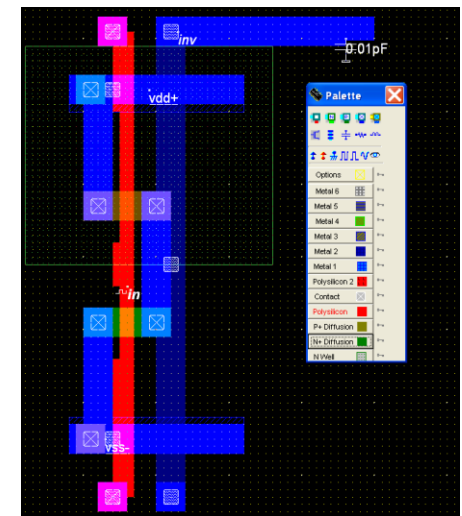
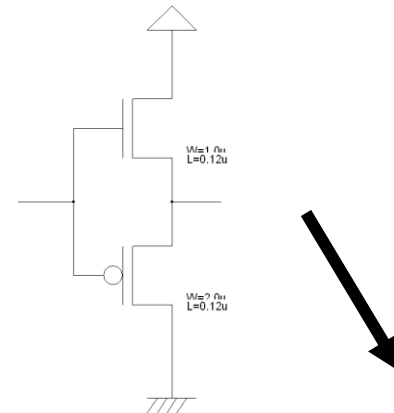
Assessment	Weight (%)	Remarks	Assessment Average (%) in academic year
Exam	60	2-hour exam	76
Coursework 1	40	Lab-based design	74

Outline

- History of IC Design
- HDL [Hardware Description Language]
- History of VHDL [Very high speed integrated circuit HDL]
- ASIC [Application Specific Integrated Circuit] and FPGA [Field Programmable Gate Array] Flow
- Intro to Programmable Logic Devices
 - FPGA Architecture
 - Spartan 3
- Intro to Simulator

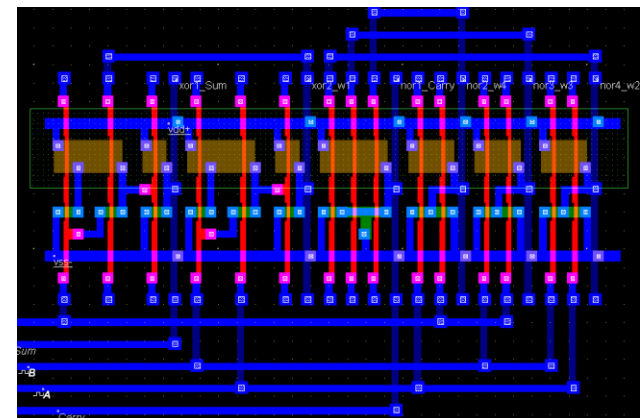
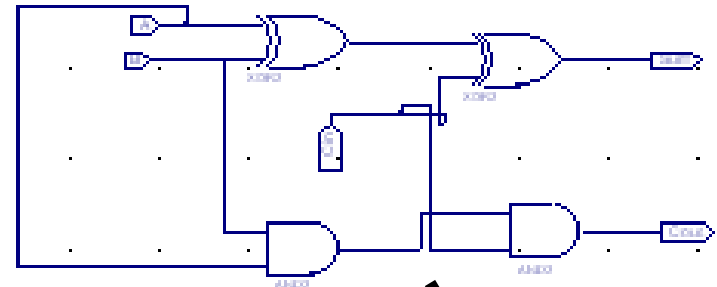
History of IC Design

- IC Design Phases Since 1970s
 - Polygon Pushing
 - Early phase of IC design 1970s
 - Designers drew the design using polygons in layout
 - Highly tedious and error prone
 - Amount of transistors can fit in a design is limited (a few thousands)



History of IC Design (cont..)

- IC Design Phases Since 1980s
 - Schematic Capture
 - Uses the Schematic Editor to capture schematic
 - Possible to design Complex Circuits
 - Design tools are available
 - Still in Use



History of IC Design (cont..)

- IC Design Phases Since 1990s until now
 - HDL
 - Uses the text to design complex circuits
 - Possible to design High Complex Circuits
 - Many Design tools are available (Modelsim, NCSim, Xilinx XST...)
 - Heavily used in IC design Industries

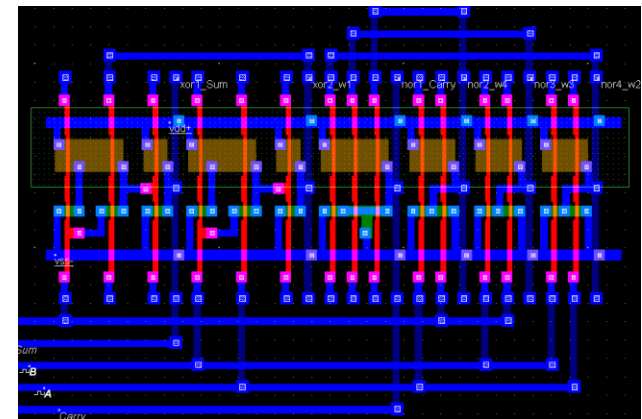
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- Uncomment the following lines to use the declarations that are
-- provided for instantiating Xilinx primitive components.
--library UNISIM;
--use UNISIM.VComponents.all;

entity full_adder_behav is
    Port ( a : in std_logic;
          b : in std_logic;
          cin : in std_logic;
          cout : out std_logic;
          sum : out std_logic);
end full_adder_behav;

architecture Behavioral of full_adder_behav is
begin

end Behavioral;
```





Hardware Description Language [HDL]

- High level languages for describing digital hardware
- Advantages of HDL
 - Flexibility: Easier to make design changes
 - Portability : Codes are technology independent
 - Document (Eg. RTC)
 - Different ways to design
 - Tools: To model, simulate and synthesis

Semiconductor manufacturing processes

- 10 μm — 1971
- 3 μm — 1975
- 1.5 μm — 1982
- 1 μm — 1985
- 800 nm (0.80 μm) — 1989
- 600 nm (0.60 μm) — 1994
- 350 nm (0.35 μm) — 1995
- 250 nm (0.25 μm) — 1998
- 180 nm (0.18 μm) — 1999
- 130 nm (0.13 μm) — 2000
- 90 nm — 2002
- 65 nm — 2006
- 45 nm — 2008
- 32 nm — 2010
- 22 nm — approx. 2011
- 16 nm — approx. 2013
- 11 nm — approx. 2015

HDL Flavors

- High level languages for describing digital hardware
- Different flavor of HDL
 - **VHDL** (Very high speed integrated circuit **H**ard **W**are **D**escription **L**anguage)
 - <http://www.eda.org/comp.lang.vhdl/>
 - **VERILOG HDL**
 - (<http://www.eda.org/vlog-synth/>)
 - Originated at Automated Integrated Design Systems (now renamed as Gateway Design Automation) in 1985
 - Become IEEE standard in 1995
 - **ABEL**
 - **A**dvanced **B**oolean **E**xpression **L**anguage
 - Created in 1983 by Data I/O Corporation
 - Now Owned by Xilinx

HDL Languages (cont...)

- Different flavor of HDL.....
 - **SLIDE** - Structured Language for Interface Description & Evaluation
 - **CONLAN** - CONsensus LANguage
 - **ISPS** - Instruction Set Processor Specification
 - **ADLIB** - A design Language for Indicating Behavior
 - **HILL** - Hierarchy Layout Language
 - **VHDL - AMS**
 - ([http:// www.eda.org/vhdl-arms/](http://www.eda.org/vhdl-arms/))
 - VHDL – Analog and Mixed Signal
 - No synthesis yet
 - **Handel - C**
 - (<http://www.celoxica.com>)
 - Based on ANSI – C

VHDL History

- In 1982, IBM, TI and Intermetrics awarded US Department of Defense (DoD) contract for specification and implementation of this new language
- More structured design methods and tools needed
- VHDL version 7.2 established in 1984
- DoD transferred all rights of the language definition to the IEEE.

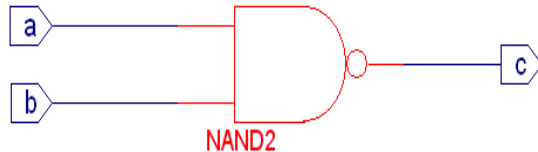
VHDL History (cont...)

- 1987 Standardized by IEEE
 - IEEE 1076-1987 (VHDL – 87)
 - The first VHDL standard
 - No signal strengths X and Z
- 1993 IEEE 1076 -1993 (VHDL – 93)
 - Appended standard IEEE 1164
 - Contains multi valued logic 0,1,X, Z etc..
- 2002 IEEE 1076 -2002 (VHDL -2002)
 - Latest VHDL standard
- Ongoing
 - IEEE 1076 – 200X
 - <http://www.eda.org/vasg/>
- 2004 VHDL RTL Synthesis
- VHDL AMS (analog and mixed signal)

IEEE VHDL Standards

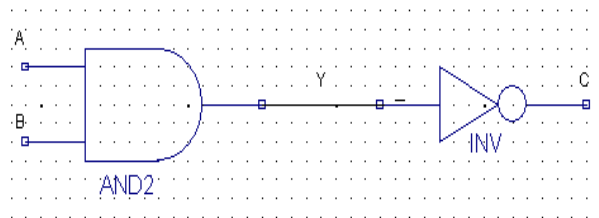
- P1076.1 (VHDL – AMS)
 - VHDL Analog and Mixed Signal extension
- P1076.2 (VHDL Math Packages)
 - Math_Real and Math_Complex
- P1076.3 (VHDL Synthesis Package)
 - Numeric_bit and Numeric_std
- P1076.4 Timing
 - VHDL Initiative towards ASIC libraries
- P1164 (VHDL Multivalue Logic)
 - Std_Logic_1164

Simple VHDL Code



```
11 entity nand_comp is
12     Port ( a : in std_logic;
13           b : in std_logic;
14           c : out std_logic);
15 end nand_comp;
16
17 architecture Behavioral of nand_comp is
18
19 begin
20     c <= A NAND B;
21
22 end Behavioral;
23
```


Simple VHDL Code

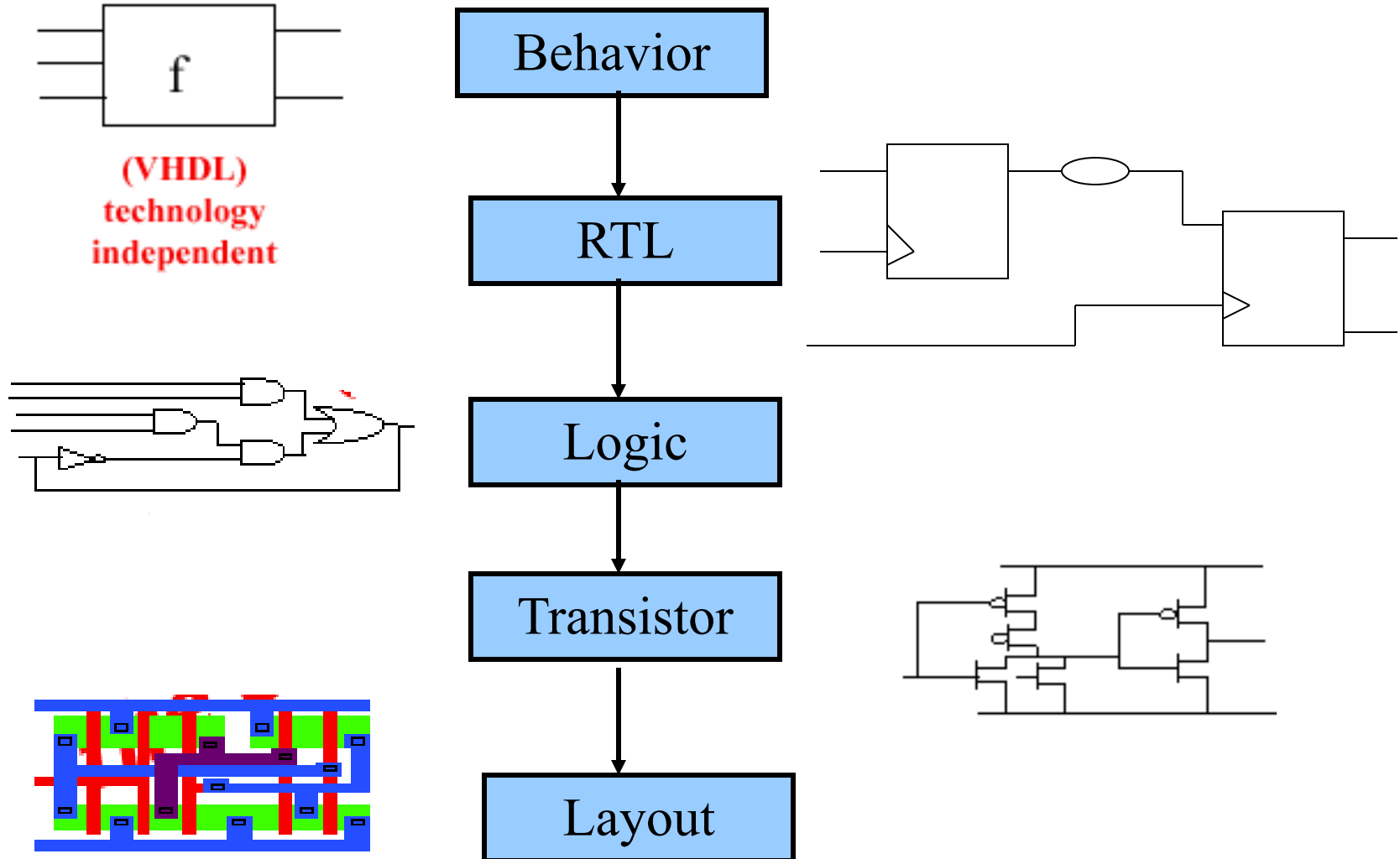


```

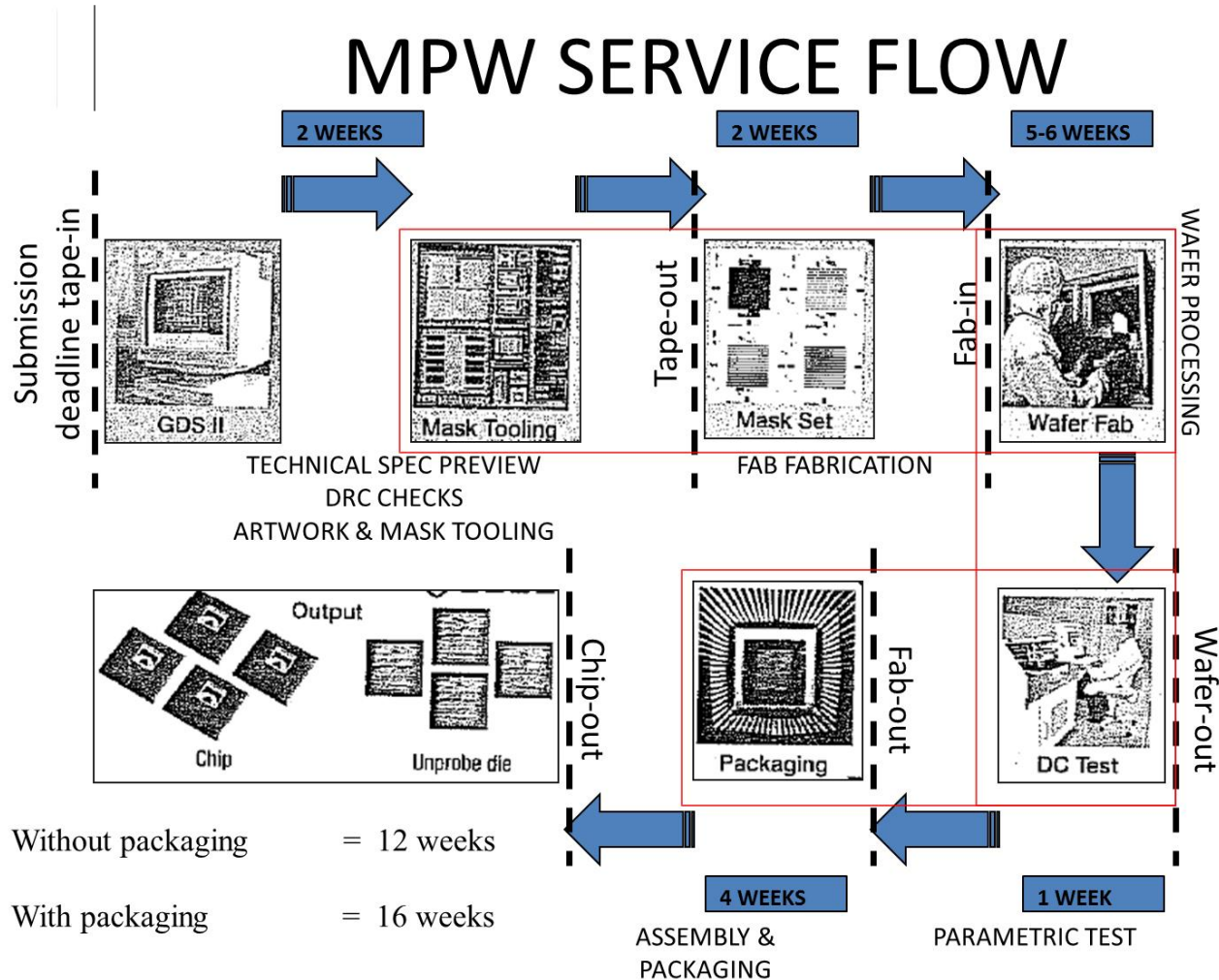
11 entity nand_comp is
12     Port ( a : in std_logic;
13           b : in std_logic;
14           c : out std_logic);
15 end nand_comp;
16
17 architecture Behavioral of nand_comp is
18     signal y;
19 begin
20
21         Y <= A AND B;
22         C <= NOT Y;
23
24 end Behavioral;
  
```

ASIC Flow

Application Specific Integrated Circuit

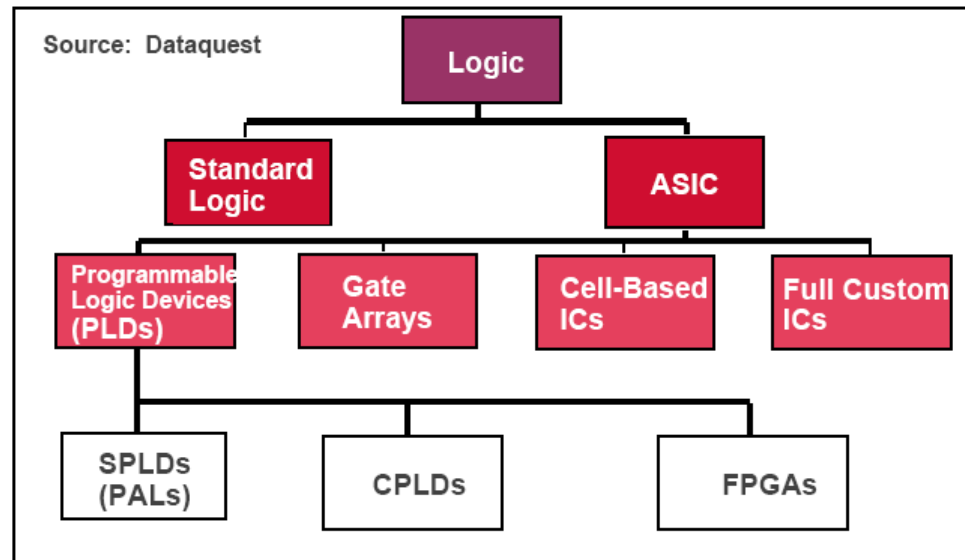


ASIC Flow



Ref: MIMOS

HDL Implementation



Acronyms

SPLD = Simple Prog. Logic Device

PAL = Prog. Array of Logic

CPLD = Complex PLD

FPGA = Field Prog. Gate Array

Common Resources

Configurable Logic Blocks (CLB)

- Memory Look-Up Table
- AND-OR planes
- Simple gates

Input / Output Blocks (IOB)

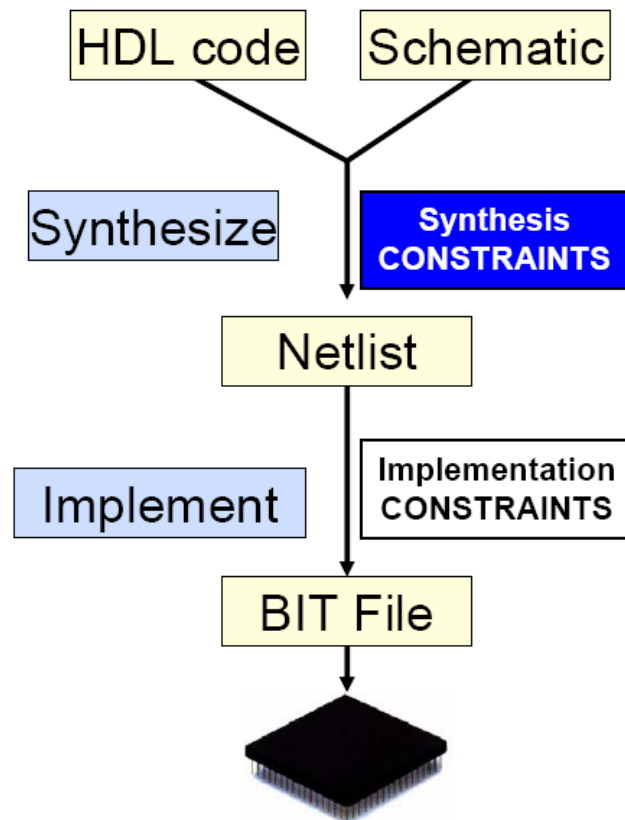
- Bidirectional, latches, inverters, pullup/pulldowns

Interconnect or Routing

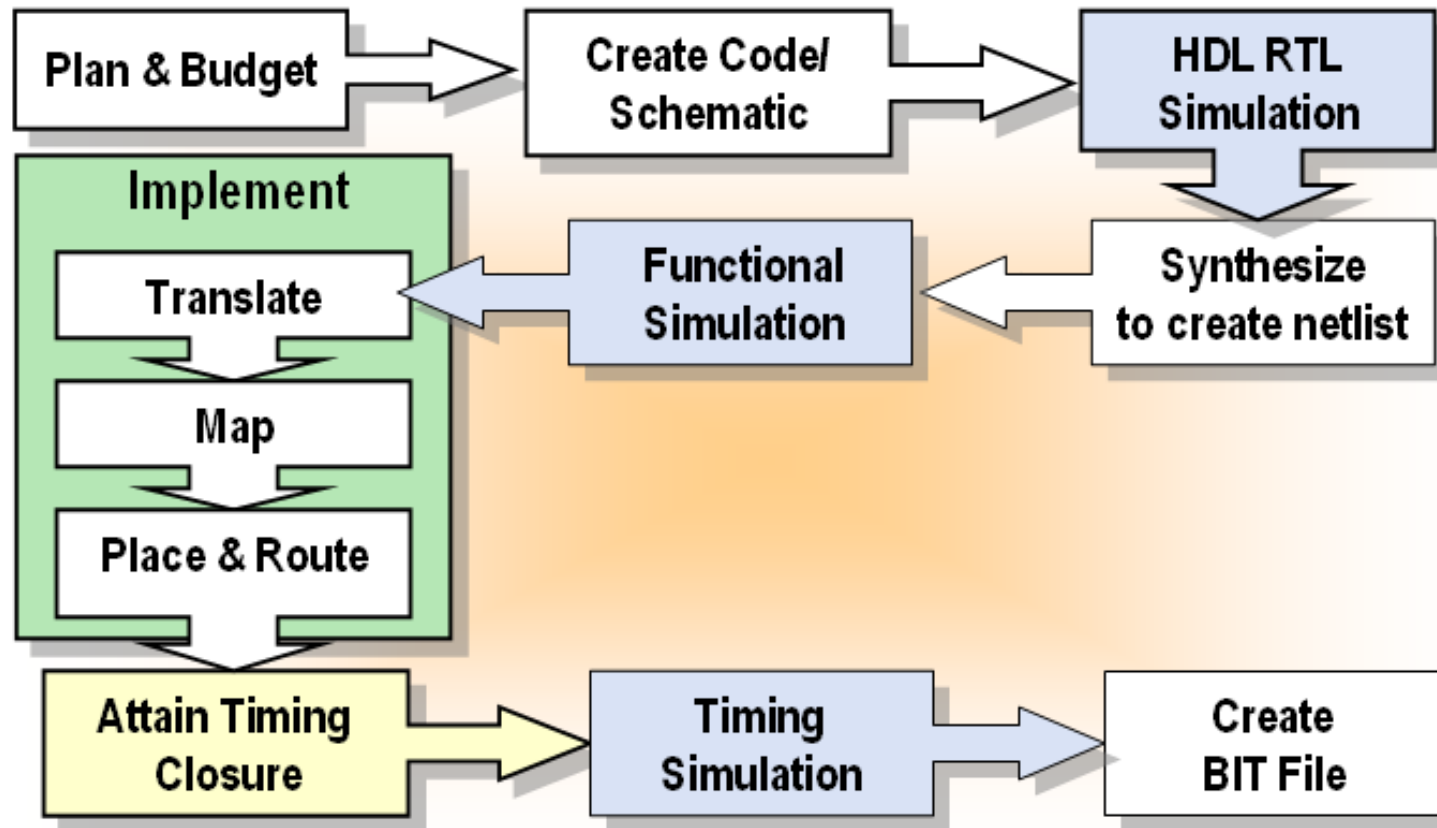
- Local, internal feedback, and global

Adopted from Jim Duckworth

Xilinx FPGA Flow



Xilinx FPGA Flow

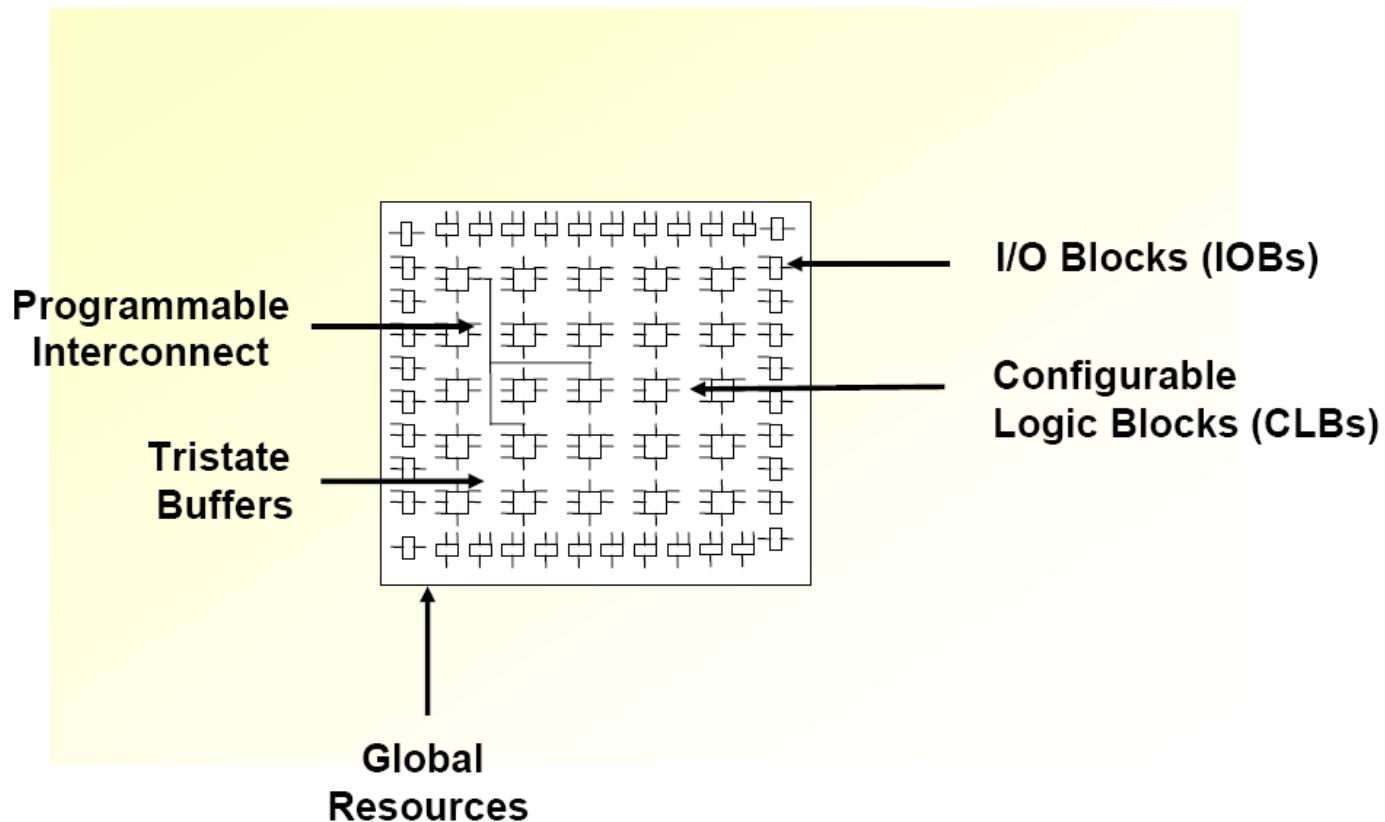


Adopted from Xilinx.com

HDL Implementation

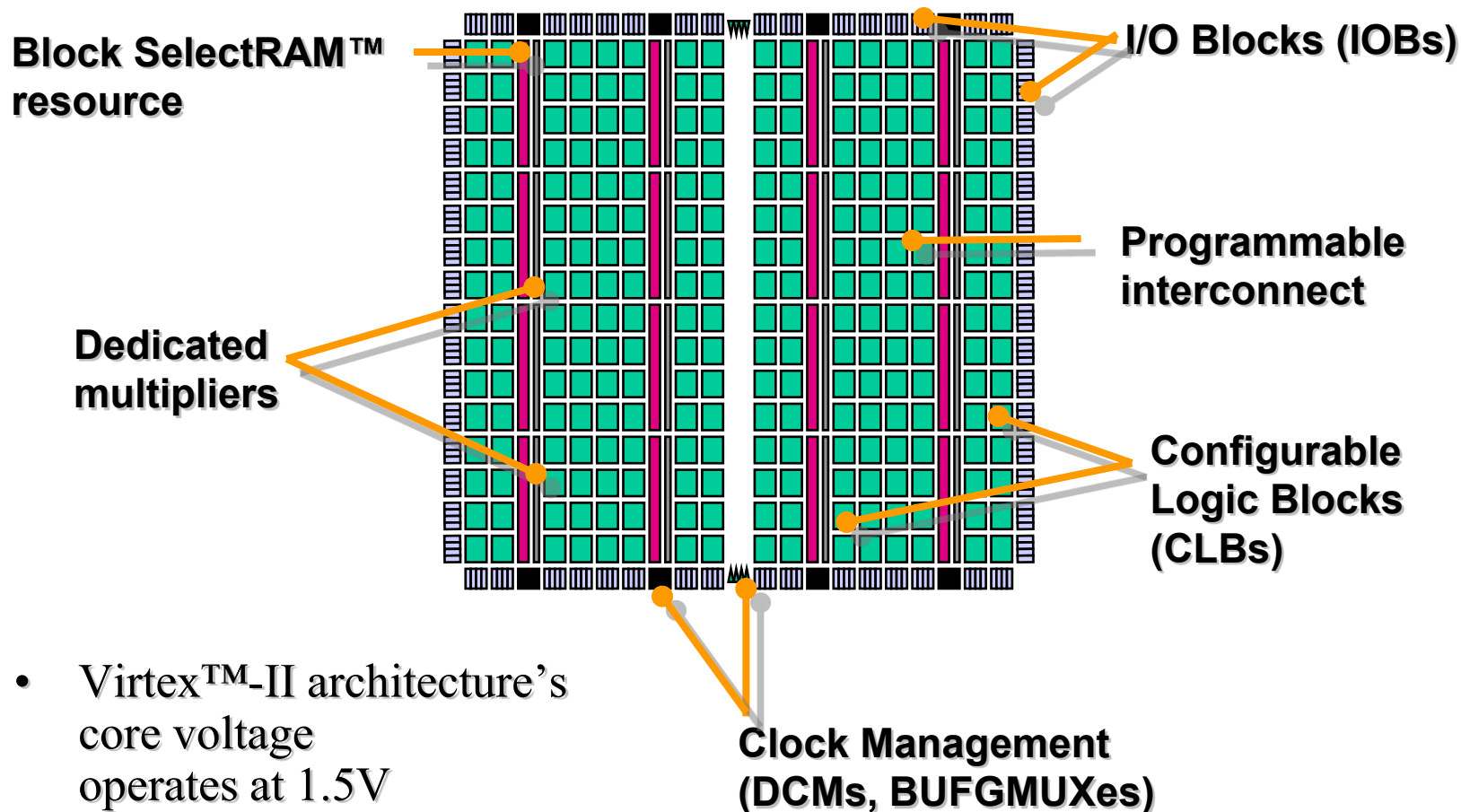
- FPGA
 - Field Programmable Gate Array
 - Low cost for low volume
 - Reprogrammable and quick time to market
- ASIC
 - Application Specific Integrated Circuits
 - High Design & Fabrication cost
 - Slow time to market
 - Low cost for large volume

Spartan 3 FPGA Architecture



Adopted from Xilinx.com

Xilinx Virtex-II Architecture



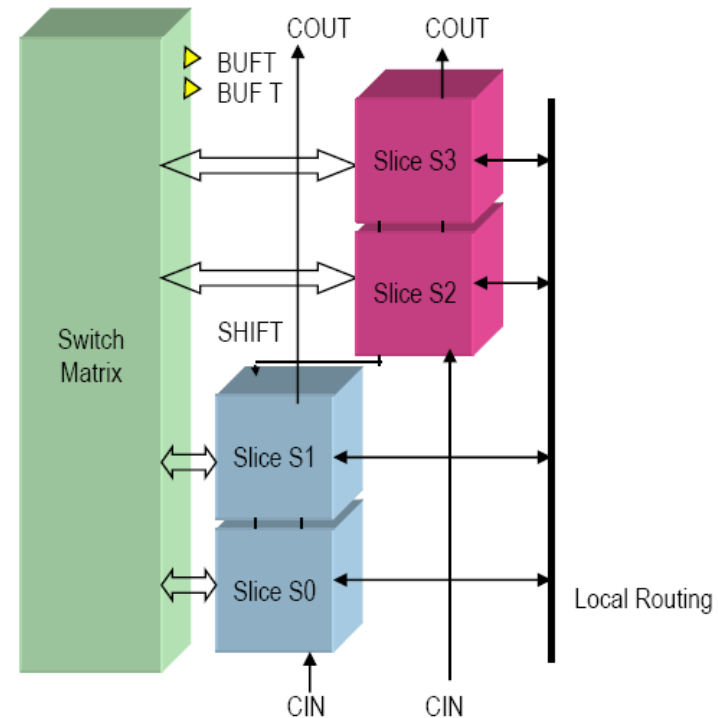
SRAM FPGA Architecture

- Configurable Logic Blocks (CLBs)
 - RAM-based look-up tables to implement logic
 - Storage elements for flip-flops or latches
- Input/Output Blocks
 - Supports bidirectional data flow and 3-state operation
 - Supports different signal standards including LVDS
 - Double-data rate registers included
 - Digitally controlled impedance provides on-chip terminations
- Block RAM provides data storage
 - 18-Kbit dual-port blocks
- Multiplier blocks (accepts two 18-bit binary numbers)
- Digital Clock Manager (DCM)
 - Provides distribution, delaying, mult, div, phase shift of clocks

Adopted from Jim Duckworth

CLB

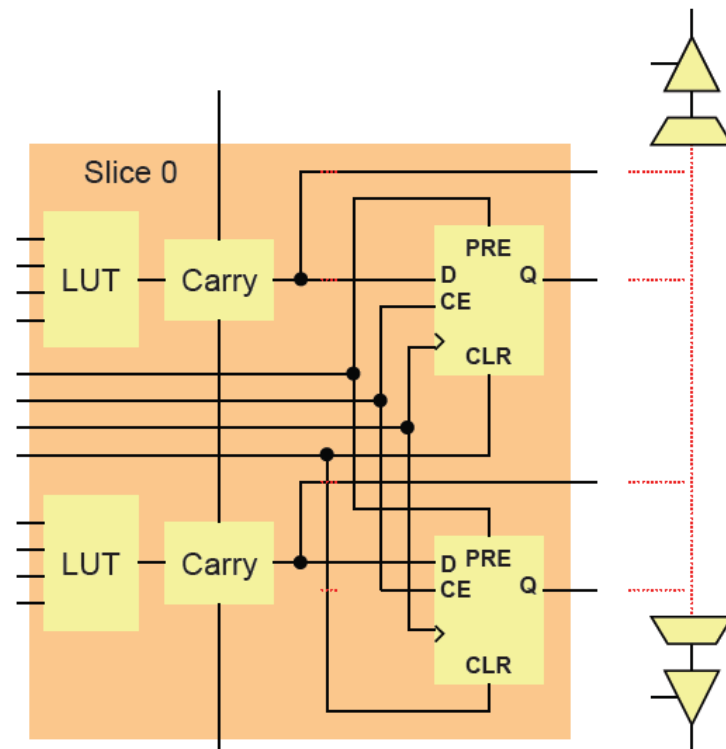
- Each Virtex™-II CLB contains four slices
 - Local routing provides feedback between slices in the same CLB, and it provides routing to neighboring CLBs
 - A switch matrix provides access to general routing resources



Adopted from Xilinx.com

Slice

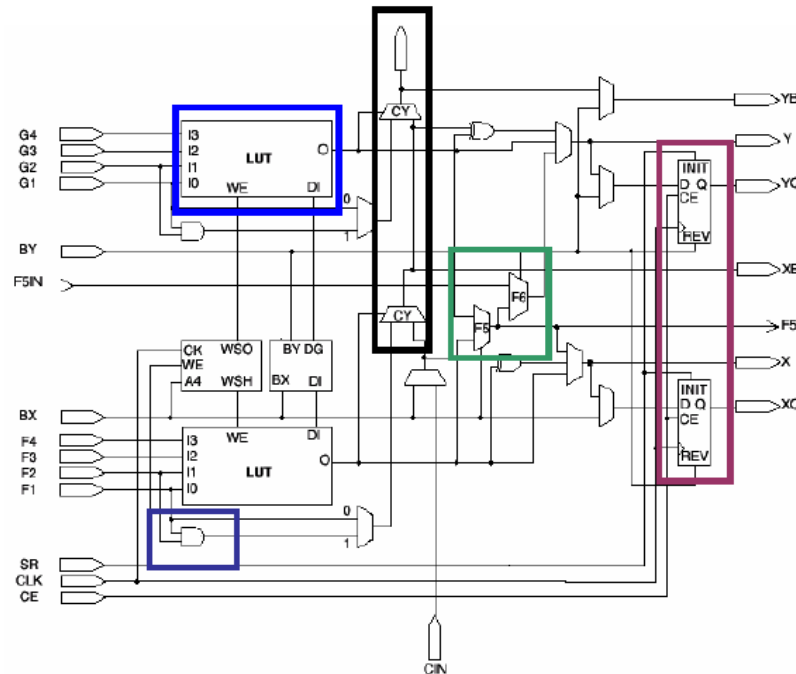
- Each slice has four outputs
 - Two registered outputs, two non-registered outputs
 - Two BUFTs associated with each CLB, accessible by all 16 CLB outputs
- Carry logic runs vertically, up only
 - Two independent carry chains per CLB



Adopted from Xilinx.com

Slice

- The next slides will discuss the slice features
 - LUTs
 - MUXF5, MUXF6, MUXF7, MUXF8 (only the F5 and F6 MUX are shown in the diagram)
 - Carry Logic
 - MULT_ANDs
 - Sequential Elements

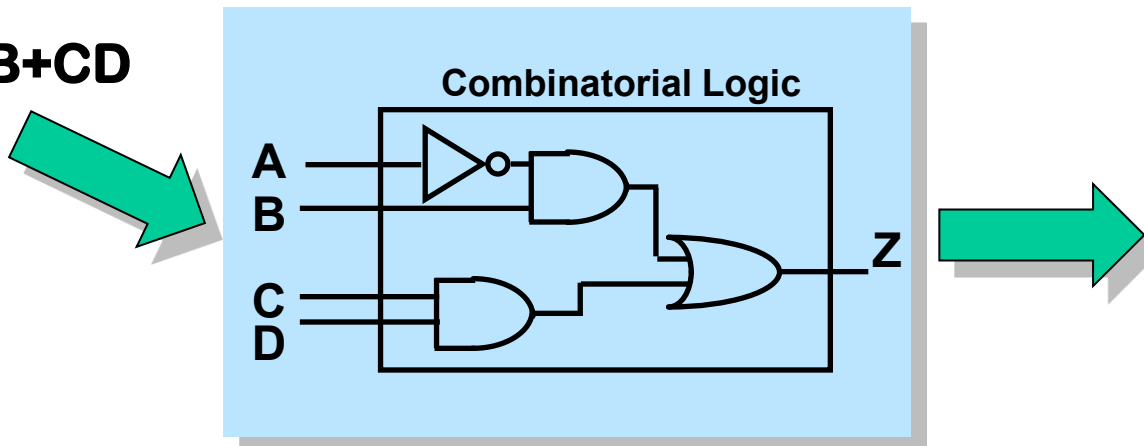


Adopted from Xilinx.com

Look-Up Tables

- Combinatorial logic is stored in Look-Up Tables (LUTs)
 - Also called Function Generators (FGs)
 - Capacity is limited by the number of inputs, not by the complexity
- Delay through the LUT is constant

$$Z = A'B + CD$$

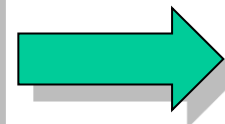
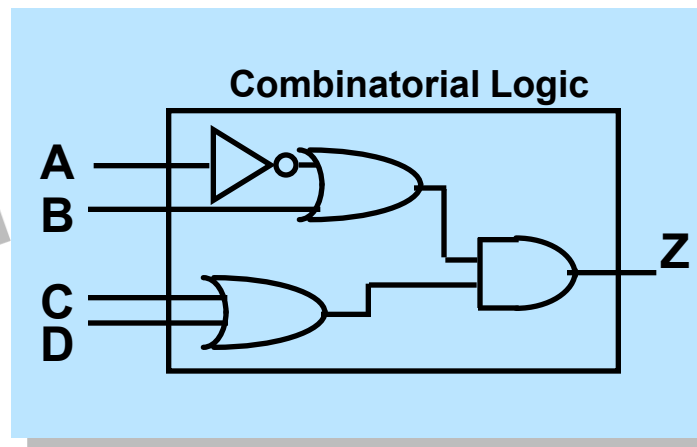
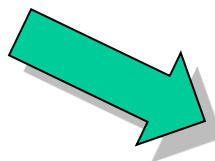


A	B	C	D	Z
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Look-Up Tables

- Combinatorial logic is stored in Look-Up Tables (LUTs)
 - Also called Function Generators (FGs)
 - Capacity is limited by the number of inputs, not by the complexity
- Delay through the LUT is constant

$$Z = (A' + B) \cdot (C + D)$$

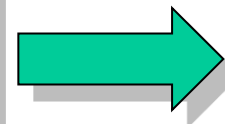
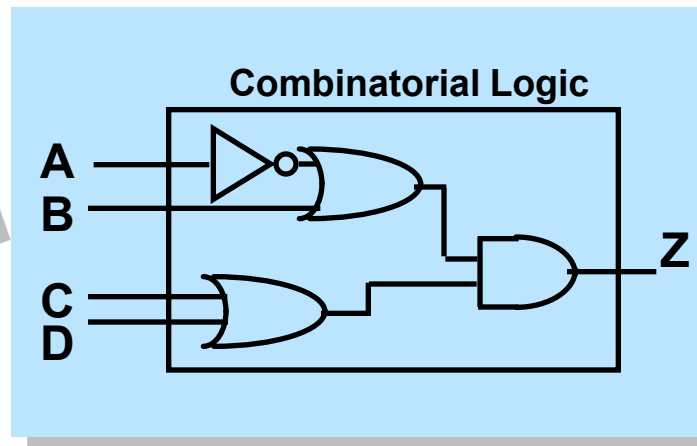
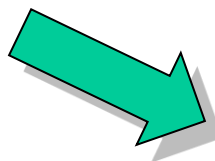


A	B	C	D	Z
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Look-Up Tables

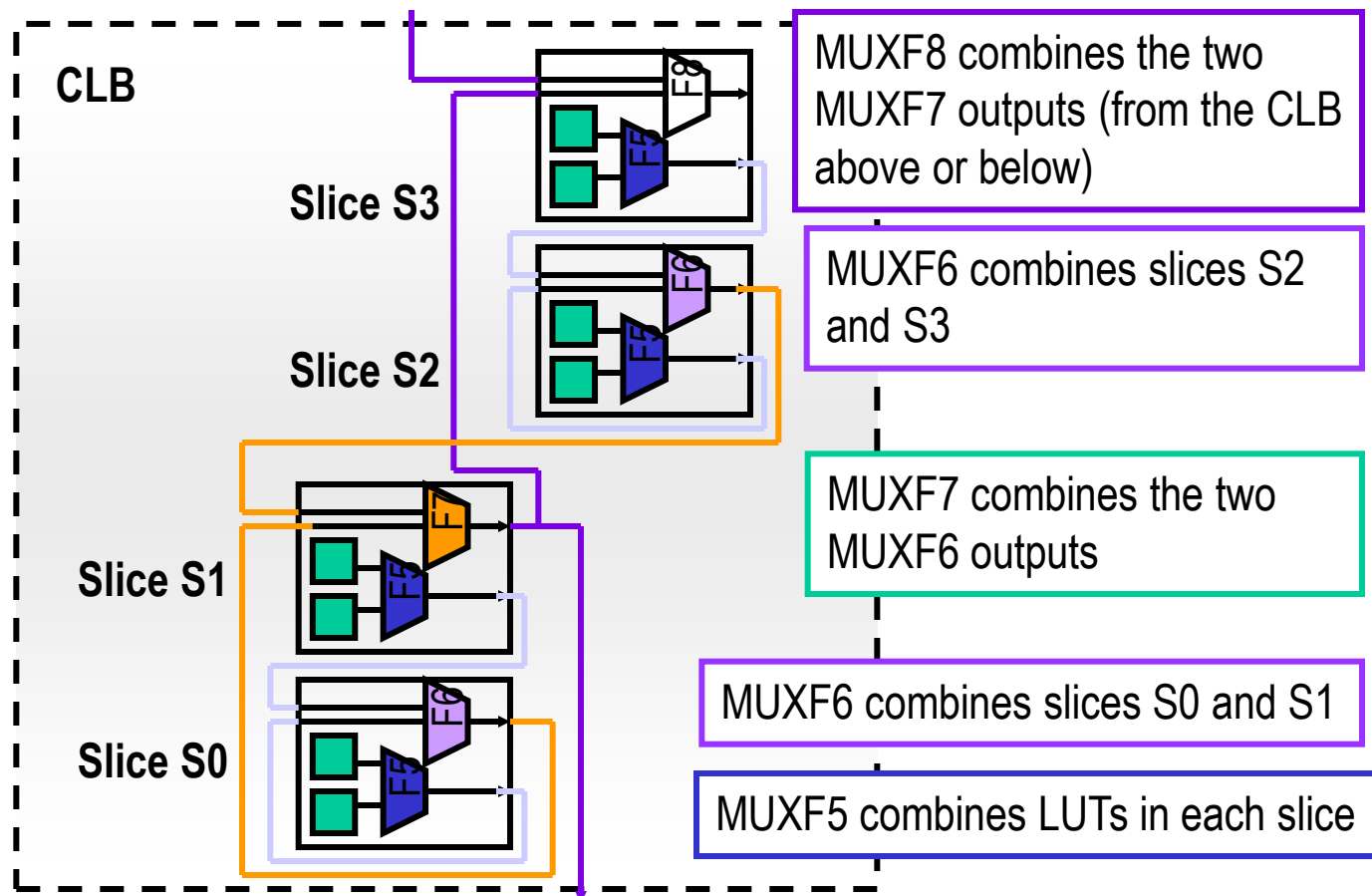
- Combinatorial logic is stored in Look-Up Tables (LUTs)
 - Also called Function Generators (FGs)
 - Capacity is limited by the number of inputs, not by the complexity
- Delay through the LUT is constant

$$Z = (A' + B) \cdot (C + D)$$



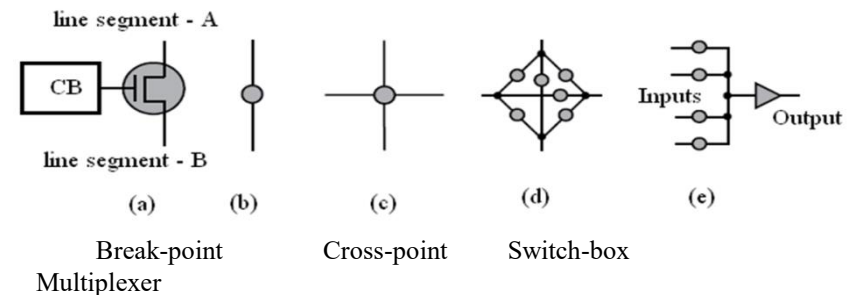
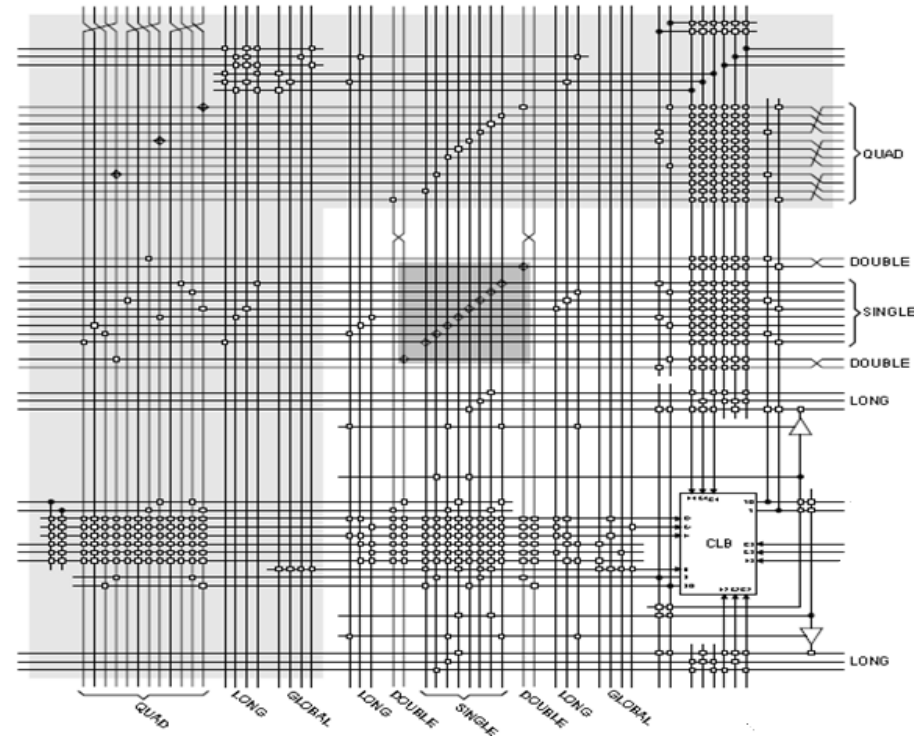
A	B	C	D	Z
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Connecting Look-Up Tables

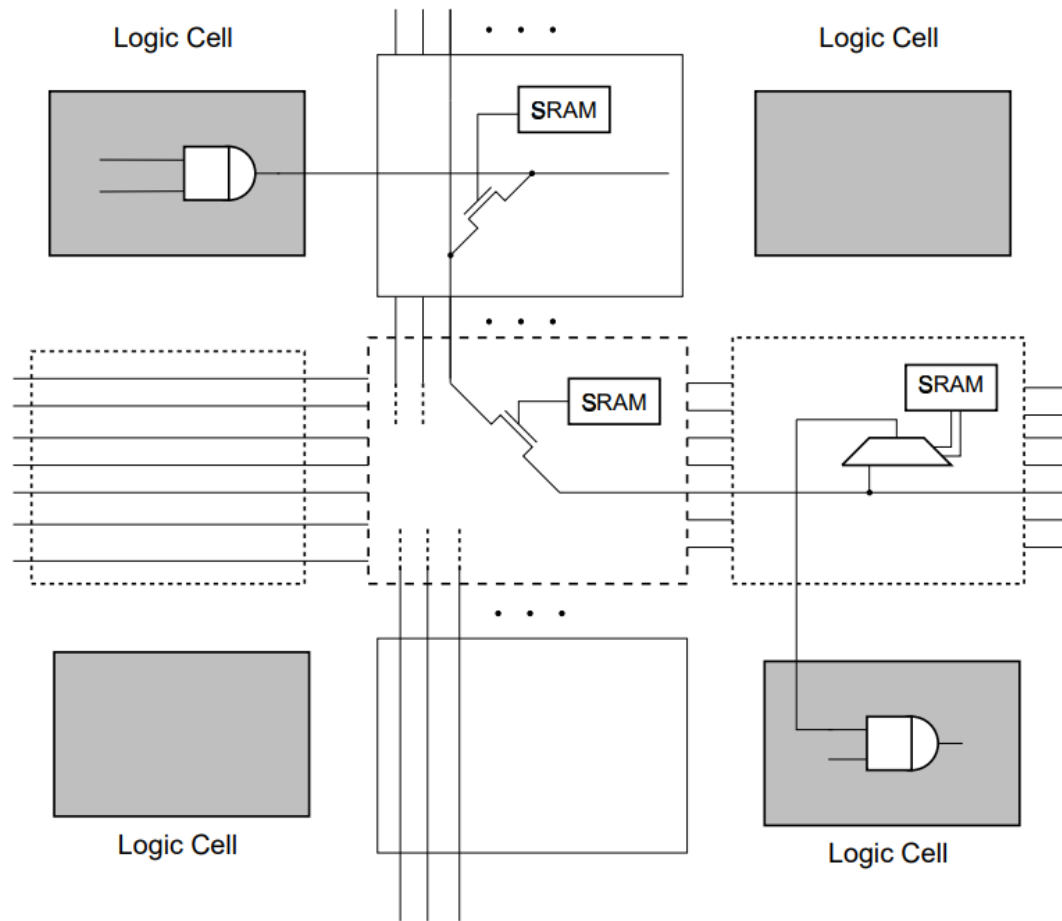


Programmable Interconnect

- Programmable Interconnect
 - made of many nets
- Nets
 - varying lengths of horizontal and vertical line segments
- Line segments
 - metal segments (wires)
- Programmable Interconnect Points (PIPs)
 - switches to connect or disconnect line segments
- Desired routing
 - Connecting the different types of line segments using appropriate PIPs



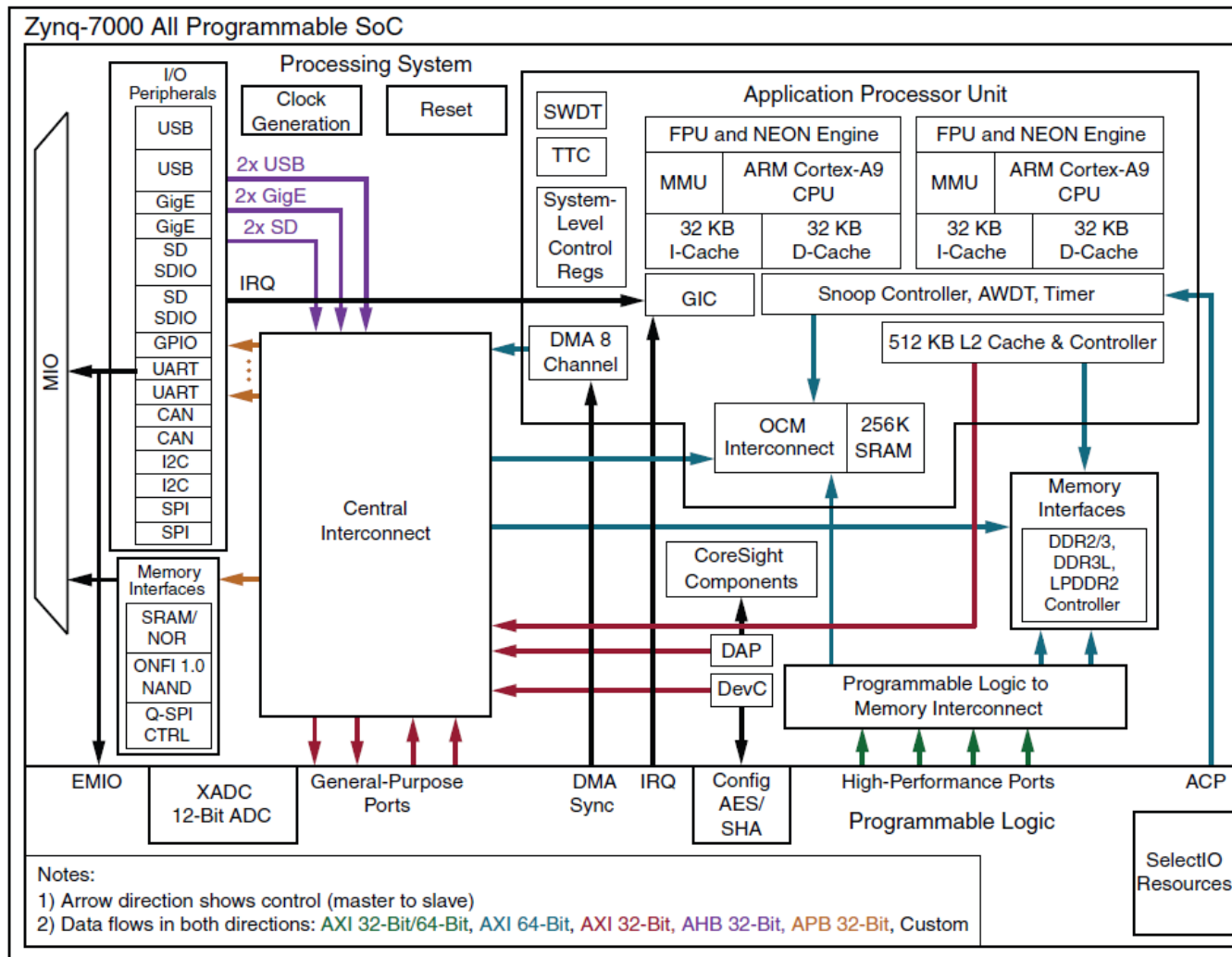
Programmable Interconnect



Spartan 3 FPGA

Device	System Gates	CLBs (4 slices)	CLB flip-flops	Block Ram (bits)	User IO
XC3S200	200K	480	3,840	216K	173
XC3S1000	1M	1,920	15,360	432K	391
XC3S4000	4M	6,912	55,296	1,728K	712

All Programmable SOC

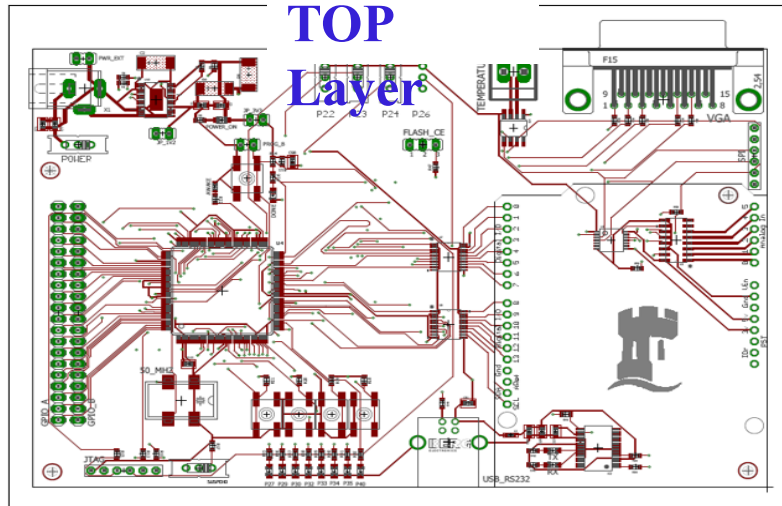


Customized FPGA Board

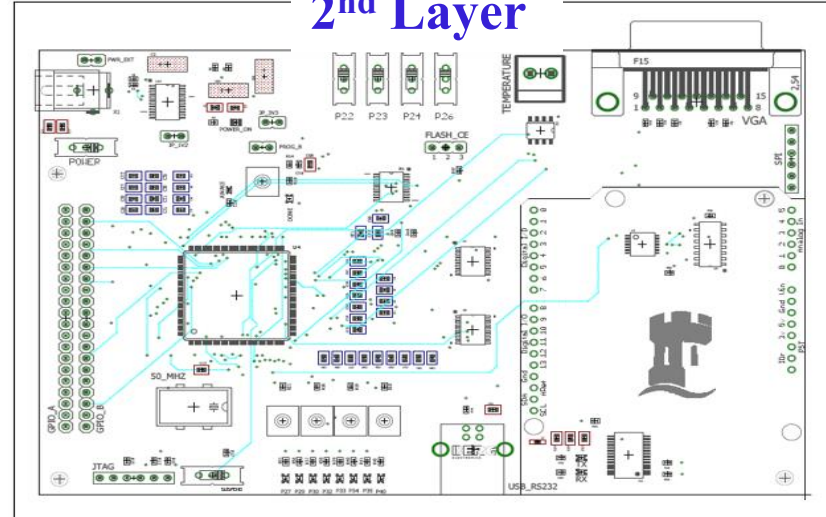


University of
Nottingham
UK | CHINA | MALAYSIA

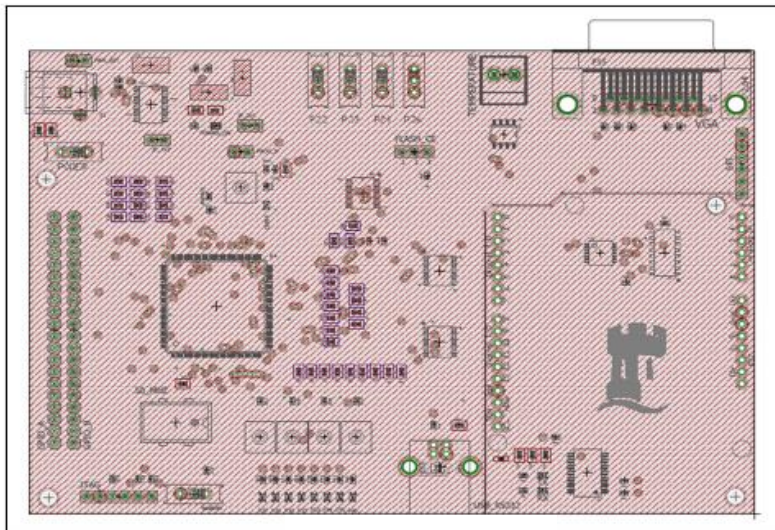
**TOP
Layer**



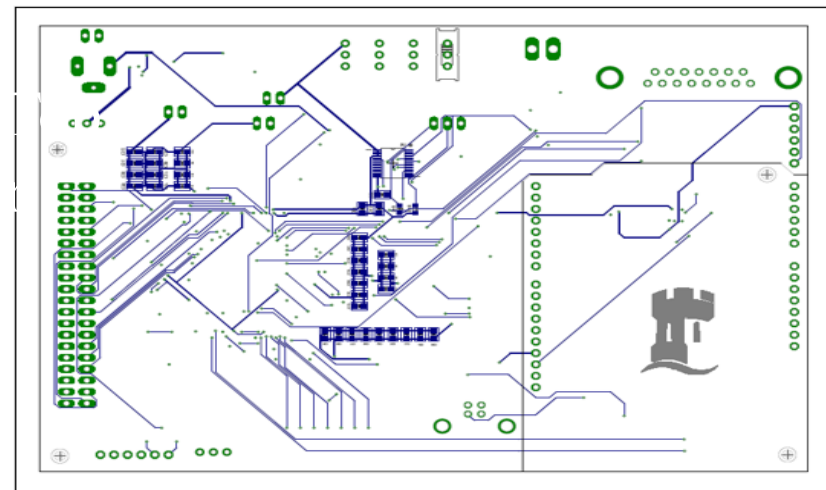
2nd Layer



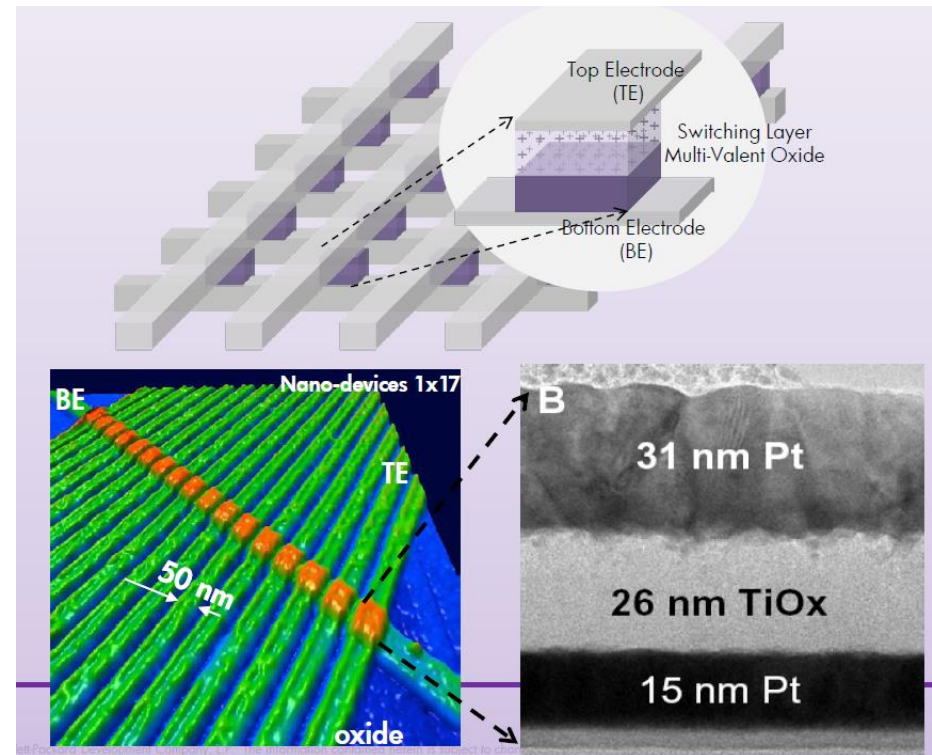
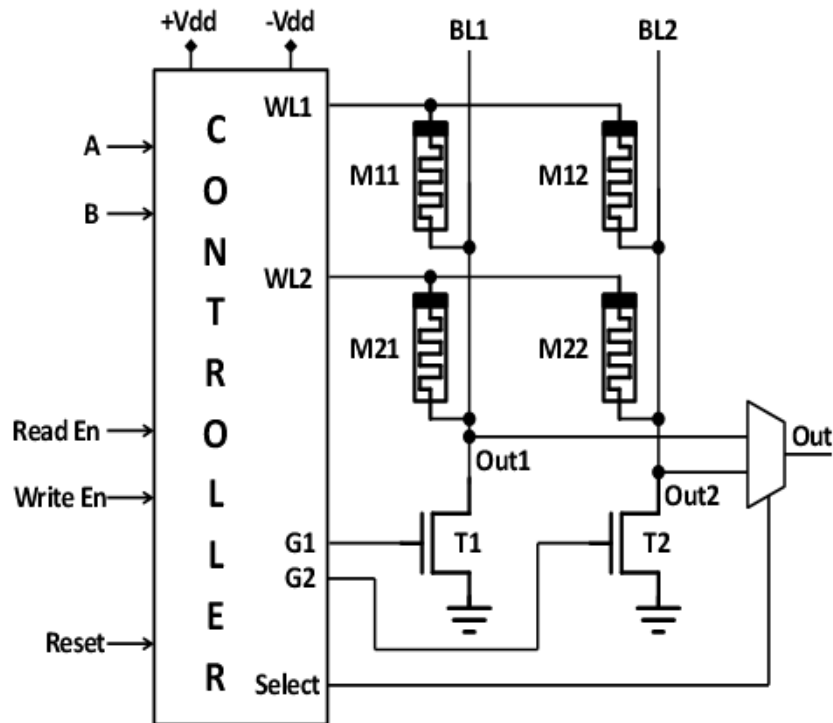
3rd Layer



Bottom Layer

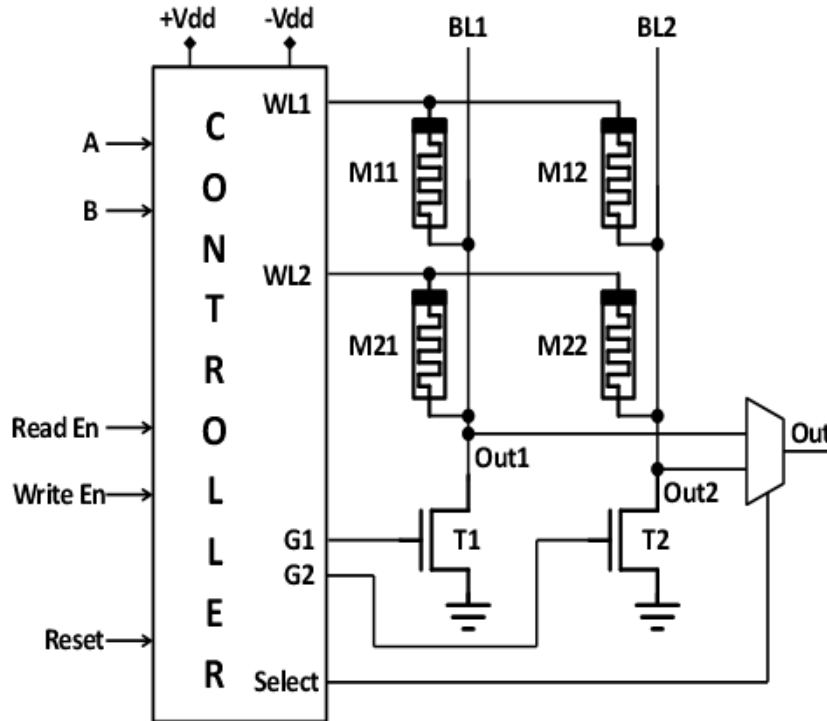


Non- Volatile LUT



Adopted from HP

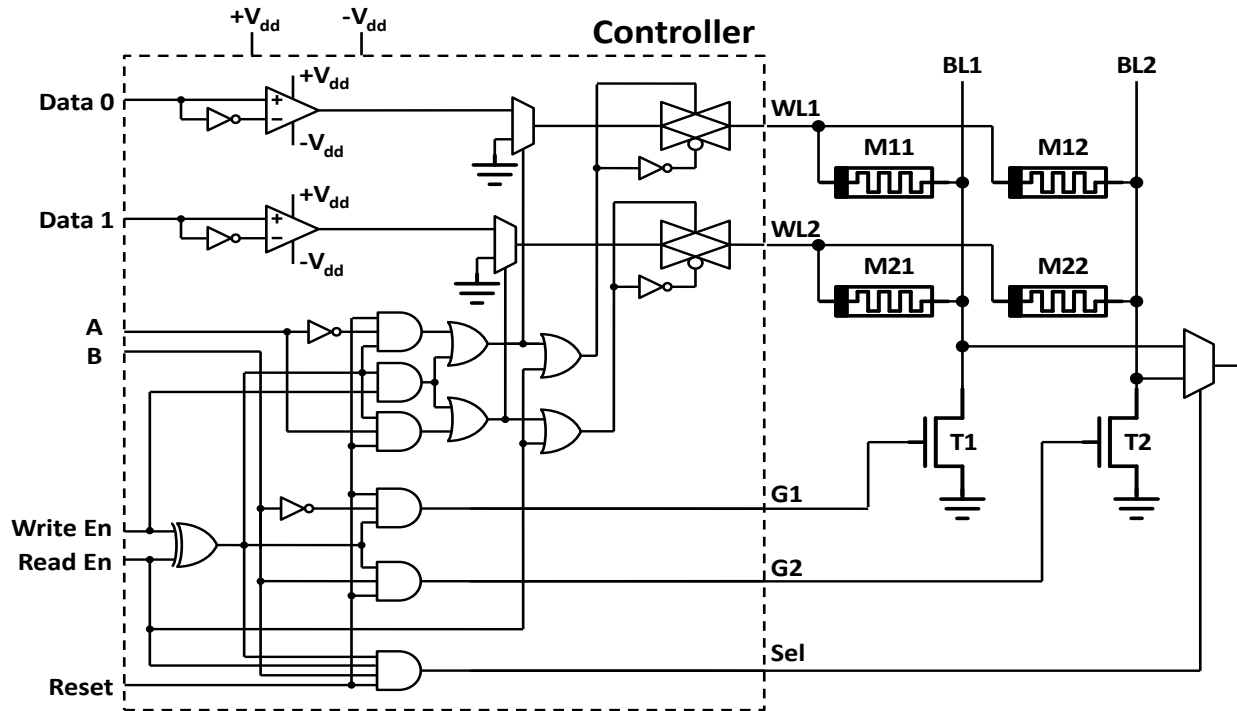
Non- Volatile LUT



- New WRITE and READ methods
- Transistors control to selected/unselected memristors.
- No Write Data monitoring circuit
- Restore pulse only for READ 0 operation
- No Refresh pulse
- Fast access time
- No power dissipation during standby mode

	WL1 Voltage	WL2 Voltage	BL1 Voltage	BL2 Voltage
Write 1	Vdd	Floating	GND	Floating
Write 0	-Vdd	Floating	GND	Floating
Read	+Vdd	GND	To load	To load
Restore	-Vdd	GND	To load	To load

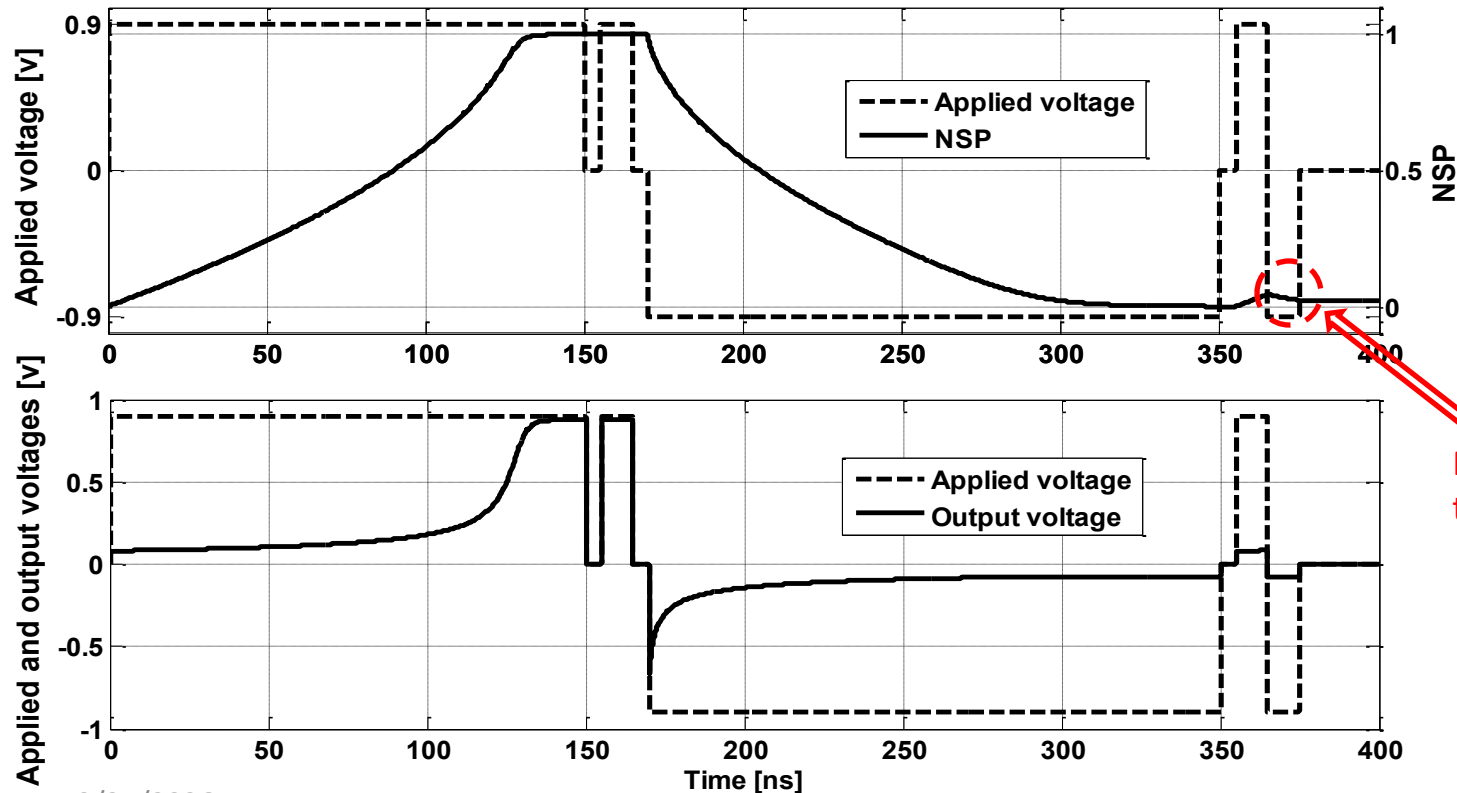
Non-Volatile IIT Controller



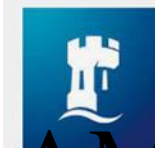
A	B	WL1	WL2	G1	G2	Sel
0	0	Data0	Z	1	0	0
0	1	Data0	Z	0	1	1
1	0	Z	Data1	1	0	0
1	1	Z	Data1	0	1	1

One Bit NV LUT Simulation

WRITE 1, READ 1, WRITE 0, READ 0 and RESTORE operations



NSP Not reaching
to original value



Simulation Results-NV & SRAM

LUT

Virtex 4 FPGA (XC4VLX160)															
Benchmark Circuit	2 Input LUTs	3 Input LUTs	4 Input LUTs	WRITE OPERATION						READ OPERATION					
				Average DEALY (ns)			Average EDP (pJ.ns)			Average DEALY (ns)			Average EDP(pJ.ns)		
				Imp-LUT	LUT	SRAM	Imp-LUT	LUT	SRAM	Imp-LUT	LUT	SRAM	Imp-LUT	LUT	SRAM
298	4	6	11	5105.18	11443.12	3.917	7.30E+6	17862481	0.38903	9.53E-4	0.1137	17.168	8.64E-5	0.452	0.452
400	11	14	19	10771.22	22960.73	7.938	2.98E+7	69873405	1.50597	20.4E-4	0.2313	33.293	3.76E-4	1.691	1.691
510	13	17	55	19853.86	47936.33	16.186	1.24E+8	3.26E+08	7.18203	36.01E-4	0.4672	75.274	1.34E-3	8.761	8.761
820	10	26	72	26071.76	62885.94	21.238	2.14E+8	5.6E+08	12.352	47.3E-4	0.6131	98.692	2.32E-3	15.05	15.05
953	25	33	123	41975.38	103235.7	34.744	5.72E+8	1.52E+09	33.6919	75.5E-4	1.0016	163.822	6.06E-3	41.56	41.56
1238	27	41	155	51920.42	128498.1	43.198	8.84E+8	2.37E+09	52.3959	93.2E-4	1.2448	204.626	9.29E-3	64.88	64.88
1488	26	48	183	60149.66	149829.4	50.312	1.19E+9	3.23E+09	71.5095	107.6E-4	1.4491	239.448	1.25E-2	88.89	88.89
5378	65	96	206	88277.76	202309.5	68.973	2.26E+9	5.65E+09	123.559	163.4E-4	1.9989	307.866	2.60E-2	145.82	145.82
15850	140	270	376	197380	428570.5	147.639	1.03E+10	2.46E+10	532.564	372.8E-4	4.296	629.474	1.27E-1	606.08	606.08
35932	1192	489	1307	652808.3	1435265	493.230	1.15E+11	2.78E+11	6031.66	1227.7E-4	14.34	2126.055	1.4019	6923.80	6923.80

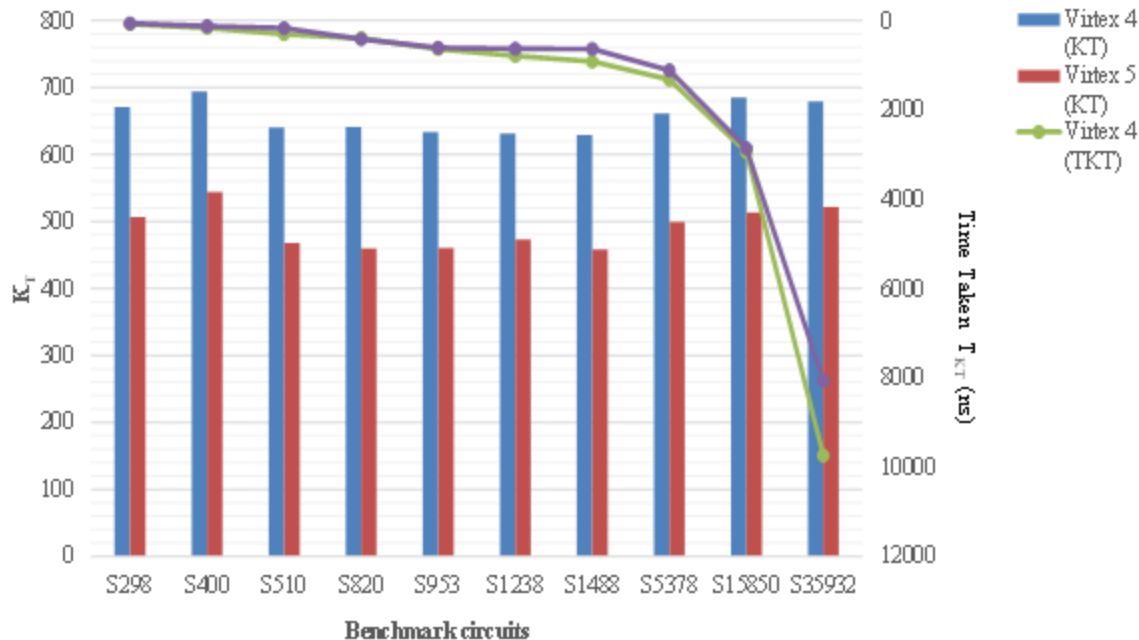
Virtex 5 FPGA (XC5VLX220)																	
Benchmark Circuit	2 Input LUTs	3 Input LUTs	4 Input LUTs	5 Input LUTs	6 Input LUTs	WRITE						READ					
						Average Delay (ns)			Average EDP (pJ.ns)			Average Delay (ns)			Average EDP (pJ.ns)		
						Imp-LUT	LUT	SRAM	Imp-LUT	LUT	SRAM	Imp-LUT	LUT	SRAM	Imp-LUT	LUT	SRAM
298	1	3	3	3	4	4321.26	14937.09	4.22	1.03E+7	38974005	0.7553	0.0007062	0.1198	29.61	6.865E-5	0.0703	1.3686
400	2	5	6	9	4	8392.16	25812.24	7.83	3.27E+7	1.09E+08	2.25	0.0014124	0.223	47.64	25.4803E-5	0.2109	3.5403
510	0	2	5	11	17	11530.82	49140.09	12.99	9.78E+7	4.52E+08	8.35	0.00169488	0.3640	105.30	50.4874E-5	0.7648	17.382
820	4	3	7	25	47	27919.1	124419.93	32.167	6.09E+8	2.95E+09	53.31	0.00402534	0.899	271.46	296.785E-5	4.87	115.528
953	13	9	10	31	71	41887.28	183517.56	47.32	1.34E+9	6.41E+09	115.22	0.00614394	1.325	399.73	666.669E-5	10.54	250.254
1238	9	9	19	39	60	43276.46	178134.89	47.52	1.31E+9	5.89E+09	109.36	0.00649704	1.334	377.64	708.452E-5	10.04	223.397
1488	11	10	24	24	76	43376.28	192626.61	49.38	1.46E+9	7.09E+09	126.85	0.00632049	1.382	421.63	715.602E-5	11.60	278.452
5378	32	41	49	59	80	79173.02	283224.22	79.18	3.65E+9	1.42E+10	272.80	0.01274691	2.240	569.75	2320.16E-5	25.31	506.961
15850	99	99	96	197	163	201061.16	684171.86	196.81	2.18E+10	8.06E+10	159.16	0.0328383	5.580	1339.15	14864.35E-5	148.31	1800.221
35932	1152	326	380	129	542	608074.92	1897444.42	541.06	1.79E+11	6.13E+11	11733.93	0.10536504	15.451	3650.18	131411.44E-5	1105.20	20665.076

9/24/2026

EEEE4076-TNK

44

Delay Performance Analysis



$$T_{O,I,B} = t_{W,I,B} + K_T t_{R,I,B}$$

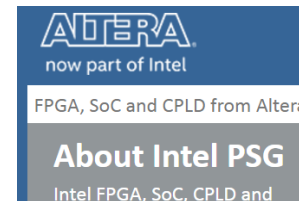
$T_{O,I,B}$ is average delay time for operation O (either READ, R or WRITE, W) using scheme I (I=SRAM or I=MEM) for benchmark B
Single WRITE followed by K_T consecutive READ

Read operations are performed more often than WRITE operations in an FPGA

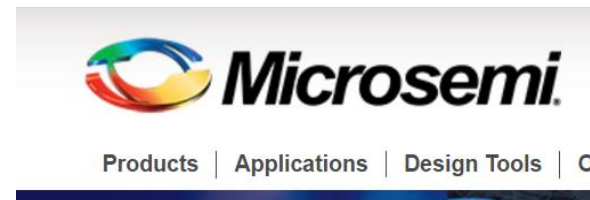
FPGA Devices

- Xilinx FPGA www.xilinx.com

- Altera FPGA www.altera.com



- Actel FPGA www.actel.com



SoC

Zynq-7000
Zynq-7000Q
XA Zynq-7000

FPGA

UltraScale
Virtex UltraScale
Kintex UltraScale
7 Series
Artix-7
Artix-7Q
Kintex-7
Kintex-7Q
Virtex-7
Virtex-7Q
Virtex-6
Virtex-6Q
Virtex-5

Virtex-4

Virtex-4Q

Virtex-4QV

Spartan-6

Spartan-6Q

Spartan-3A DSP

Extended Spartan-3A

Spartan-3AN

Spartan-3A

Spartan-3E

Spartan-3

CPLD

CoolRunner-II

XC9500XL

FPGA Applications



Aerospace and Defense

Xilinx serves the aerospace and defense industry with commercial, industrial, military, and space grade products.



ASIC Prototyping

ASIC prototyping with FPGAs enables fast and accurate SoC system modeling and verification of embedded software.



Audio

Xilinx FPGAs and targeted design platforms enable higher degrees of flexibility, faster time-to-market, and lower overall non-recurring engineering costs (NRE) for a wide range of video and imaging applications.



Automotive

Xilinx Automotive (XA) is the programmable engine for many of the infotainment, driver assistance, and driver information systems of today, and the next-generation systems of tomorrow.



Broadcast

Adapt to changing requirements faster and lengthen product life cycles with Broadcast Targeted Design Platforms and solutions for high-end professional broadcast systems.



Consumer Electronics

The demands and dynamics of the consumer electronics market are ever changing and growing.



Data Center

Designed for high-bandwidth, low-latency servers, networking, and storage applications to bring higher value into cloud deployments.



High Performance Computing

Advancements in process technology and device architecture have allowed Xilinx FPGAs to be used in high-performance computing (HPC) applications acting as bridges and switches, as accelerators offloading portions of a software application running on the CPU, or as fixed function hardware accelerators supporting high throughput data processing.



Industrial

Xilinx All Programmable platforms and solutions provide levels of performance and systems integration that go beyond typical microprocessor, DSP and ASIC capability for factory automation and industrial imaging applications.



Medical

For diagnostic, monitoring, and therapy applications, the Virtex FPGA and Spartan® FPGA families can be used to meet a range of processing, display, and I/O interface requirements.



Security

Xilinx offers solutions that meet the evolving needs of security applications, from access control to surveillance and safety systems.



Smarter Networks

The biggest driver in the communications and networking markets is the insatiable need for bandwidth as traffic explodes well beyond the capabilities of networks to support that traffic.



Smarter Vision

Xilinx Smarter Vision technologies provide developers with meaningful market relevant tools that accelerate the development of Smarter Vision products.



Wired Communications

Xilinx Wired Communications solutions are designed to meet Service Providers' needs to provide "connected any-where, any-time, any-device" services.



Wireless Communications

From basestations to microwave backhaul to satellite communications, Xilinx is enabling faster, more cost-efficient wireless communications worldwide and advancing equipment manufacturers ahead of the competition.

Compilers & Simulators

- **All VHDL files need to be compiled before they can be simulated.**
- **Many compilers exist**
 - <http://www.eda.org/comp.lang.vhdl/FAQ3.html>
- **FPGA specific simulators**
 - XilinxFoundation tools
 - Altera Maxplus II
- **Generic VHDL simulators**
 - ModelSim(The market leader)
- Xilinx and Altera, have incorporated ModelSim into their latest FPGA tools

More on Modelsim

- **Model Technology, Mentor Graphics Corporation**
 - <http://www.model.com>
 - <http://www.mentor.com>
- **ModelSim XilinxEdition**
 - ModelSimXE III 6.0a
 - Integrated into XilinxISE 7.1i software
 - <http://www.xilinx.com/ise/mxe3/download.htm>
- **ModelSim AlteraEdition**
 - ModelSim-Altera6.0c Software
 - Integrated into QuartusII version 5.0 software
 - https://www.altera.com/support/software/download/eda_software/modelsim/msm-index.jsp

Link to download Xilinx

Link to download Xilinx 14.7 ISE:

https://numcmy-my.sharepoint.com/:f:/g/personal/efyma2_nottingham_edu_my/ErdNeOaMAYVLsuDIB0PjwKEBad7XmxYXiM1XpTddmOA64Q?e=KgfOlX

YouTube tutorial for Windows 10 ISE 14.7 installation workaround: <https://www.youtube.com/watch?v=VMEIPCjqinA>

Why VHDL?

- Formal specification language
- Reduces paperwork
- Technology and Process independence
- Design re-use
- Alternative to schematics
- Can make a very quick start
- Wide range of levels gate, RTL, behaviour
- Description and simulation of complex systems
- Microprocessors

Remember

- Hardware description language is not similar as other programming language
- FPGA doesn't have a build in processor to execute the VHDL code
- Synthesis tools infers the behaviour described by the HDL code and prepares a hardware design
- FPGA is configured by the BIT file
- Need to understand combinational and sequential logic

Exercise 1

True/False

1. Does design implemented using ASIC method yield low cost for low volume?
2. VHDL handles logic gates only?
3. Is ASIC a reprogrammable flow?
4. VHDL code is technology independent?

Summary